TRABAJO PRÁCTICO

Modalidad: grupal

Producto: Sistema de reserva de asientos

Descripción del sistema

El sistema permite **reservar asientos para eventos**. Es un dominio abstracto, no atado a ningún negocio en particular: un evento puede ser un recital, una conferencia o una función. Lo único que importa es que tiene una cantidad limitada de asientos y que muchas personas pueden intentar reservar el mismo asiento al mismo tiempo.

Las reservas pasan por diferentes estados:

Estado	Significado
Retenida	El asiento está apartado hasta que se procesa el cobro. Tiene un tiempo de vencimiento
Confirmada	El cobro fue exitoso. El asiento queda reservado
Rechazada	La reserva no se concretó. Se registra el motivo: asiento ocupado, pago fallido o pago no disponible
Vencida	Se agotó el tiempo de retención sin un resultado de pago

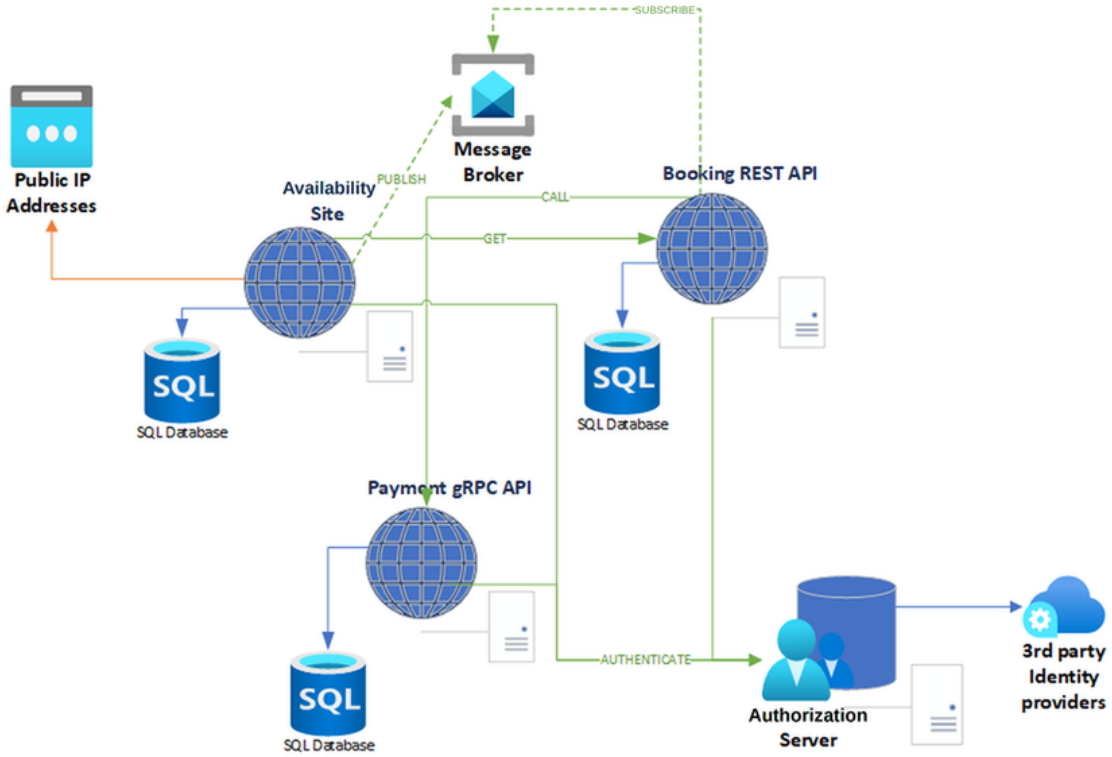
Un asiento está **ocupado** mientras tenga una reserva retenida o confirmada. En cualquier otro estado, el asiento vuelve a estar disponible.

Flujo de una reserva

1. **Consulta:** Cualquier visitante consulta en Availability los asientos disponibles de un evento. No requiere iniciar sesión.
2. **Autenticación:** Para reservar, el usuario obtiene un token del servidor de autorización.
3. **Solicitud:** El usuario envía a Availability la solicitud de reserva de un asiento. Availability verifica que el asiento exista, registra la solicitud con un identificador único y la publica en el broker. Responde de inmediato que **la solicitud fue recibida**, sin esperar el resultado.
4. **Procesamiento:** Una de las réplicas de Booking toma el mensaje. Si el asiento está libre, crea una reserva retenida. Si está ocupado, la rechaza.
5. **Cobro:** Booking llama por gRPC a Payment para cobrar la reserva retenida.
6. **Resultado:** Según la respuesta de Payment, Booking confirma la reserva o la rechaza.

7. **Seguimiento:** El usuario consulta el resultado de su solicitud a través de Availability, que a su vez lo consulta a Booking.
8. **Vencimiento:** Si una reserva retenida no llega a un resultado dentro de su tiempo de vencimiento, pasa a vencida y el asiento queda liberado.

Diagrama de arquitectura:



Glosario de dominio:

- **Evento:** la ocasión con asientos para reservar
- **Mensaje:** lo que viaja por el broker
- **Suceso:** acciones dentro de los componentes del sistema
- **Asiento**: Lugar físico para sentarse
- **Reserva**: La renta de un asiento físico
- **Pago**: Finalización de la renta de un asiento físico

Asignación:

1) Complete la tabla de interpretación del diagrama, justifique

Componente	Rol	Justifique la participación en el sistema
Availability	Consulta disponibilidad y recibe solicitudes de reserva	Es la puerta de entrada para consultar y solicitar asientos
Booking	Verifica disponibilidad, crea y actualiza reservas	Es el responsable del estado de la reserva

Payment	Procesa y aprueba/rechaza pagos	Determina si una reserva puede confirmarse
Message Broker	Transporta solicitudes entre Availability y booking	Permite comunicacion asincrona y desacopla servicios
Authorization	Emite y valida tokens	Protege las llamadas entre usuarios y microservicios
Notification	Informa al usuario cuando la reserva se confirma	Cumple el requisito de notificacion posterior al pago

2) Los componentes pueden ser compuestos con modelos de lenguaje, pero es requerido pasar un historial de prompts realizados para implementar cada componente por integrante del grupo. Puede usar Copilot para generar un archivo de memoria con los prompts generados y entregar adjunto el archivo de memoria con nombre y apellido por integrante del grupo junto a su repositorio.

3) Implemente los componentes del diagrama. Justifique en código como comentarios las decisiones o patrones que implemente.
    a) El sitio Availability:
        i) Seguirá una arquitectura de software Clean architecture
        ii) Dominio anémico
    b) Booking REST API:
        i) Seguirá una arquitectura de software N-Layer
        ii) Dominio anémico
    c) Payment gRPC API:
        i) Seguirá una arquitectura de software N-Layer
        ii) Dominio anémico
    d) El estilo arquitectónico es Microservicios
    e) Los microservicios deben autenticar entre sí con el servidor de autorización

f) Agregar logs de auditoría para identificar el inicio y fin de una cadena de sucesos en cada microservicio
g) Señalar en el diagrama donde el sistema se vuelve eventualmente consistente.
h) Agregar al diagrama y tabla un servicio de notificaciones que informe al usuario cuando la reserva está creada y confirmada una vez que el pago fue realizado con éxito
i) Realizar un diagrama de arquitectura de software de cada API
j) La solución final debe compilar y ejecutar un flujo de Happy path de reserva de asientos y uno de fallo en el sistema si un componente falta.

Los entregables se enviarán en un repositorio de GitHub por mensaje privado desde la plataforma TUP con el siguiente formato:
Entrega de TP Grupo: N°, Link: {link_de_repo_publico}